

Geo-Aware WebAssembly (WASM) Serverless Orchestrator

Bachelor Thesis Defense

Cao Tr`n Khánh Linh — 23BI14252

University of Science and Technology of Hanoi
Department of Information and Communication Technology

External Supervisor: Eng. Tr`n Tuấn Anh   Internal Supervisor: Assoc. Prof. Tr`n Giang Sn

Thesis Defense
July 2026

Presentation Overview

- ① Introduction & Motivation
- ② System Design
- ③ Evaluation & Results
- ④ Conclusion & Perspective

Context: Edge Computing Meets Serverless

Edge computing

- Moves computation close to where data is produced and consumed
- Critical for latency-sensitive workloads: IoT gateways, real-time filtering, privacy-sensitive preprocessing
- Key question: not only *how much* capacity, but *where* it is located

Serverless computing

- Users submit *functions*, not servers
- The platform selects the environment, executes, and returns the result
- Natural fit for the edge: developers should never pick a node manually

This thesis

wasmCat: a native Go master-worker orchestrator that runs WebAssembly functions on geo-distributed edge workers — with no Docker, Kubernetes, or external WASM runtime.

Why Not Container-Centric Orchestration?

- ❶ **Startup cost** — image pull, layer unpacking, namespace and cgroup setup dominate short function-style executions
- ❷ **Installation complexity** — Docker / containerd / K8s control planes are heavy for small edge nodes and prototypes
- ❸ **Scheduling semantics** — cluster schedulers optimize resource availability and affinity, not *geographic proximity* to the user

Gap

Lightweight, request-driven edge functions need: fast sandboxed startup, minimal node dependencies, and location-aware placement.

WebAssembly as the Edge Serverless Runtime

- **Portable:** one `.wasm` binary runs on any host, compiled from Rust, Go, TinyGo, C, Zig, ...
- **Sandboxed:** modules interact with the host only via explicit imports/exports and linear memory
- **Fast startup:** compile once, instantiate per request in microseconds — no filesystem or process preparation

Runtime choice: wazero

- Pure-Go WebAssembly runtime, embedded *inside* the worker binary
- No CGO, no external engine to install
- Whole system ships as native Go binaries

Research Problem & Contributions

Problem: design and implement a lightweight WebAssembly serverless orchestrator that *securely* dispatches requests to *geographically distributed* edge workers, without a container runtime.

Contributions:

- 1 A Go-native master-worker orchestrator with a clear control-plane / data-plane split
- 2 Capacity-aware **geo-routing** using the Haversine formula
- 3 Embedded **wazero** execution with two ABIs (custom + WASI), digest-aware module caching, and strict resource limits
- 4 **Security & reliability:** mutual TLS with certificate-bound worker identity, module digest verification, durable SQLite jobs with idempotency and recovery
- 5 **Operator tooling:** `wasmcatctl` CLI, web dashboard, JIT module provisioning from Azure Container Registry

System Architecture

architecture.png

Master Control Plane

Gateway (HTTPS API)

- Public: `/api/v1/execute`, `/api/v1/jobs`, `/api/v1/workers`, `health / ready / metrics`
- Internal: `register`, `heartbeat`, `drain`, `job-completion` callbacks
- Validates methods, body size, JSON structure, module source policy, client identity

Worker registry (in-memory)

- `map[string]WorkerNode` under `sync.RWMutex` — concurrent reads, exclusive writes
- Built from registration + 5 s heartbeats (CPU free, RAM free)
- States: `ready / draining`; background loop evicts stale workers

Design choice

Liveness is *real-time* cluster state $\Rightarrow$ in-memory registry, reconstructed from heartbeats. Durable state lives in the SQLite job store instead.

Two-stage selection

- 1 **Filter:** drop `draining` workers, then workers below
`MIN_WORKER_CPU_FREE /`
`MIN_WORKER_RAM_FREE_MB`
- 2 **Score:** great-circle distance from the user, pick the nearest eligible worker

Haversine formula ($R = 6371$ km)

$$a = \sin^2 \frac{\Delta \text{lat}}{2} + \cos(\text{lat}_1) \cos(\text{lat}_2) \sin^2 \frac{\Delta \text{lon}}{2}$$

$$d = R \cdot 2 \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

Stated honestly

Geographic distance is a *proxy* for network latency, not a measurement. Extending scheduling with measured RTT, queue depth, and cache locality is identified future work.

Dispatcher: Placement & Conservative Rescheduling

Dispatch flow (per request):

- 1 Normalize module URLs; resolve ACR references just-in-time if needed
- 2 Ask the registry for schedulable workers, the scheduler for the nearest eligible one
- 3 Forward to the worker's `/invoke` endpoint over mTLS

Conservative retry policy

- Retry **only** on transport errors and gateway-style statuses (502, 503, 504)
- Other failures are *not* replayed — user code may have already executed or failed deterministically
- Failed worker is removed from the local candidate list; the *scheduler re-runs* so capacity and location rules still decide the fallback

Just-in-Time Module Provisioning from Azure Container Registry

- 1 Dispatcher detects an `*.azurecr.io` module reference
- 2 Master exchanges its Azure identity (`DefaultAzureCredential`) for an ACR token scoped to `repository:<name>:pull`
- 3 Master fetches the OCI manifest, selects the `application/wasm` layer, and builds the blob URL `/v2/<repo>/blobs/sha256:<digest>`
- 4 Blob URL + short-lived bearer token + digest are forwarded to the worker, which simply downloads and compiles

Security property

Workers never hold Azure credentials — only a repository-scoped, pull-only token minted per request.

- Modules published with **ORAS** as OCI artifacts
- Digest travels with the request
⇒ integrity + cache correctness

Worker Data Plane: The WasmEngine

Execution path for each `/invoke`:

- 1 Enforce payload limit, acquire concurrency **semaphore** (reject fast when full — explicit overload behavior)
- 2 Apply execution timeout; fetch & compile module on cache miss
- 3 Verify SHA-256 **digest** before compiling; mismatched bytes are never cached
- 4 Instantiate a *fresh* module instance per request $\Rightarrow$ isolated linear memory
- 5 Run the module, enforce output limit, return the result

Execution timeout	5 s	Max payload / output	1 MiB
Module fetch timeout	10 s	Max module size	10 MiB
Concurrent executions	4	Cache: 128 entries / 256 MiB / 30 min TTL	

[Table](#): Default worker execution and cache limits.

Two Execution Modes: Custom ABI and WASI

wasmCat ABI (lightweight)

- Module exports `memory`, `malloc`, `run`
- Host writes payload into linear memory, calls `run(ptr, len)`
- Returns one packed `uint64`:

high 32 bits = output pointer

low 32 bits = output length

WASI mode (standard `wasip1`)

- Module exports `_start`; payload on **stdin**, result on **stdout**, exit code 0 = success
- Runs unmodified modules from Rust, Go, TinyGo, C, ...
- `wasi_snapshot_preview1` provided by `wazero`

Auto-detection

Request field `abi` $\in \{ "", \text{wasmcat}, \text{wasi} \}$. When empty: exports `run` $\Rightarrow$ custom ABI; exports `_start` $\Rightarrow$ WASI. Both modes share the digest-aware cache and per-request isolation.

Digest-aware cache

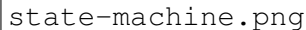
- Key = module name + URL + digest
- *Compile once* (expensive), *instantiate per request* (cheap, isolated)
- New content $\Rightarrow$ new digest $\Rightarrow$ new key — stale code is never reused
- Eviction: TTL, then max entries, then max bytes (LRU order)

Concurrency in Go

- Registry & cache guarded by `sync.RWMutex`
- Buffered channel as execution semaphore
- **In-flight compile coalescing**: first cold request becomes the leader; concurrent followers wait on a channel and reuse the cached result

Durable Jobs, Idempotency & Recovery

- `POST /api/v1/jobs` persists the request in **SQLite** (WAL mode) and returns 202 immediately
- Recovery loop leases queued jobs and dispatches them in the background
- **Idempotency**: same `request_id` + same fingerprint $\Rightarrow$ return existing state; different fingerprint $\Rightarrow$ 409 conflict



state-machine.png

Security Model

- 1 **Mutual TLS everywhere** — master and worker servers require verified client certificates before any handler runs
- 2 **Certificate-bound worker identity** — the worker ID claimed in register / heartbeat / drain / completion must match the certificate CN or DNS SAN; impersonation is rejected
- 3 **Module integrity** — SHA-256 digest verified before compilation; digest is part of the cache key
- 4 **Scoped credentials** — ACR tokens are repository-scoped, pull-only, short-lived
- 5 **Client allowlist** — optional `EXECUTE_CLIENT_ALLOWLIST` restricts who may call execution APIs

Operator interfaces preserve the boundary

The web dashboard never talks to the master from the browser — a local UI backend reuses the same mTLS client as `wasmcatctl`, so private keys are never exposed.

Experimental Setup: Multi-Region Deployment on Azure

Role	Node ID	Region	VM size
Master	wasmcat-master-dev-01	Southeast Asia	Standard_D2s_v5
Worker	worker-vn-01	Southeast Asia	Standard_D2s_v3
Worker	worker-west-us-01	West US 2	Standard_D2as_v6

Table: Testbed: Ubuntu 24.04, native binaries, one CA (key held in Azure Key Vault), regions joined by global VNet peering.

- Workload: `jsonfmt` module (canonical JSON formatting), published to ACR via ORAS *and* served from a direct HTTP URL for comparison
- Metrics: `executed_on_node_id` (routing) and `execution_time_ms` (latency), cold vs. warm cache

Result 1: Geo-Aware Routing Works End to End

Client coordinates	Location	Selected worker	Region
1.29, 103.85	Singapore	worker-vn-01	Southeast Asia
47.67, -122.12	Seattle	worker-west-us-01	West US 2

Table: Identical requests with different coordinates route to the nearest eligible worker.

- **Draining** verified: drained worker excluded from scheduling while remaining visible; traffic shifts to the remaining ready worker
- **mTLS** verified: a plain `curl` without a client certificate is rejected at the transport layer

Result 2: The Compiled-Module Cache Dominates Latency

Module source	Cold (ms)	Warm (ms)
Direct HTTP URL	2135.559	7.529
Azure ACR (OCI manifest)	2372.821	2.932

Table: Worker-reported `execution_time_ms` by module source and cache state.

- Cold path $\approx$ download + wazero compilation — paid *once* per module version per worker
- Warm path: fresh instance from the cached compiled module — a few milliseconds
- No container start, no image pull, no namespace setup on any request

Limitations (Stated Objectively)

- ① **Single active master** — control-plane availability risk; SQLite durability is local to the master host, not replicated (local recovery, *not* HA)
- ② **In-memory registry** — membership is rebuilt from heartbeats after a master restart; no persistent membership record
- ③ **Haversine $\neq$ network latency** — no visibility into RTT, congestion, or cache locality
- ④ **Coarse resource isolation** — host-level CPU/RAM telemetry; application-level limits, but no per-invocation cgroup enforcement or fuel metering
- ⑤ **Workload model** — stateless request-scoped functions only; no sockets, filesystem, or host imports beyond `wasi_snapshot_preview1`

Conclusion & Future Work

What was demonstrated

A compact, native Go system implements the essential mechanics of WebAssembly edge serverless execution: capacity- and location-aware scheduling, just-in-time ACR provisioning, mutual-TLS communication, durable and idempotent request handling — validated end to end on a two-region Azure deployment.

Future work

- **Control-plane HA:** leader election + replicated / external job store
- **Smarter scheduling:** measured RTT, queue depth, historical execution time, cache locality
- **Stronger isolation:** fuel metering, cgroup controls, admission policies for multi-tenancy
- **Wider execution model:** HTTP handler ABI, WASI filesystem/env policy, component model & WASI-HTTP

Thank You

Questions?

wasmCat — Geo-Aware WebAssembly Serverless Orchestrator
Cao Tr`n Khánh Linh — 23BI14252